

QuantSim Trading Platform

A High-Performance Virtual Trading Simulator for Portfolio Project

Aidan Dawson

Make today matter

1 Project Aim

The aim of this project is to build a high-performance, real-time trading simulation platform where users can practice investment strategies using virtual currency. The system will prioritize low-latency data processing, realistic market behavior, and an intuitive interface - serving as a comprehensive portfolio piece demonstrating full-stack engineering skills.

2 Introduction

QuantSim is a browser-based sandbox trading environment that streams live or near-live market data to users. Participants receive virtual funds to build and manage portfolios across multiple sectors (e.g., tech, energy, finance). Unlike paper trading spreadsheets, QuantSim provides real-time price charts, instant trade execution, and detailed transaction logs. The platform is designed from the ground up for performance, using modern C++ for backend services to minimize latency and maximize concurrent user handling.

3 Background & Motivation

Traditional finance education often relies on theory or delayed market simulators, leaving learners unprepared for real-world volatility. This project addresses that gap by offering hands-on, risk-free experience. From a technical perspective, QuantSim will challenge me to:

- Design efficient relational and time-series databases.
- Integrate multiple live financial APIs (REST + WebSocket).
- Build an interactive frontend with real-time visualizations.
- Implement low-latency server logic in C++.
- (Optional) Create rule-based or lightweight ML agents for automated trading.

Important: This project is built as an independent portfolio piece. All architecture and implementation decisions are my own.

4 System Requirements

4.1 Core Requirements (Must-Have)

- **Real-Time Data Visualization:**
 - Price chart for selected asset (line/candlestick).
 - Timeframe selection (1D, 1W, 1M, 3M).
 - Display current price, daily high/low, and volume.
- **Intuitive UI/UX:**
 - Clean, responsive design (mobile/desktop).
 - Dashboard showing most popular holdings and market movers.
- **Portfolio Management:**
 - Virtual starting balance (e.g., \$100,000 equivalent).

- Real-time portfolio valuation and P&L.
- **Simulated Trading Engine:**
 - Buy/sell at current market price (no slippage by design).
 - Detailed trade log (timestamp, asset, quantity, price, total).
- **Caching:** Cache API responses where permitted to reduce latency and stay within rate limits.

4.2 Optional Features (Planned for Extensibility)

- Support for multiple international markets (US, EU, Asia).
- Daily/weekly performance reports sent via email.
- Price alerts (notify when asset hits target price).
- Leaderboard comparing virtual returns among users.

4.3 Wow Factors (Stretch Goals)

- **AI Portfolio Assistant:**
 - Analyze current portfolio and suggest rebalancing based on momentum/trends.
 - Provide plain-English explanations for suggestions.
- **Automated Trading Agent:**
 - User-configurable bot that trades based on simple strategies (e.g., moving average crossover).
 - Agent logs every decision (buy/sell/hold) with reasoning.
 - Sandbox limits on max funds allocated to agent.
- **Predictive hints:** Simple statistical forecast (e.g., linear regression on recent prices) displayed on chart.

4.4 Architectural Requirements

- The system shall handle concurrent users and API requests without blocking (C++ async I/O or thread pool).
- Target 99.9% uptime during demo periods.
- Backend load distribution via multiple worker threads (no external orchestrator required).
- No real payment processing — all transactions are virtual.

4.5 Design Requirements

- User data stored in a relational database (PostgreSQL/MySQL) with a separate service for profile updates.
- Stock price graph must support dynamic time ranges and multiple assets.
- Decouple API fetching service from trading logic (clean separation of concerns).

4.6 Delivery Requirements

- Fully responsive web application.
- Deployed on a free/hobby cloud provider (Render, Fly.io, or similar) or self-hosted with public URL.
- Source code available on GitHub with clear README for running locally.

4.7 Constraints

- API rate limits - implement request throttling and local caching.
- **No slippage:** All trades execute exactly at quoted price (virtual constraint).
- Follow each API's terms of use for data caching.
- No external LLM APIs for AI features - any intelligence is hand-coded (simple heuristics, rule-based systems, or basic statistical models). This ensures full ownership of the logic.
- **Performance constraint:** Backend must handle at least 50 concurrent simulated trades per second on modest hardware.

5 Technology Stack (My Choice)

Why Not Python?

While Python is productive for prototyping, it introduces higher per-request latency and lower throughput under load due to the GIL and interpreter overhead. For a real-time trading simulator, even virtual trades should feel instantaneous. A compiled language like C++ provides predictable low latency, efficient memory usage, and true parallelism.

Proposed Stack

- **Backend:** Modern C++ (C++20/23) with a lightweight HTTP/WebSocket library (e.g., Drogon, uWebSockets, or Beast). REST API for trading actions, WebSocket for live price feeds.
- **Database:** PostgreSQL (relational data) + optional Redis for real-time caching.
- **Frontend:** React + TypeScript + TailwindCSS. Charting via Lightweight Charts or Recharts.
- **Build & Deployment:** CMake, Docker, free-tier cloud (Render/Fly.io) or VPS.

Justification

C++ allows me to write a single-threaded event loop or a multi-threaded server with fine-grained control over latency. It also integrates well with WebSocket libraries, giving true real-time updates. This choice makes the project stand out as a high-performance system rather than a typical Python/Django demo.

6 Budget

Instructions: This project has no external sponsor. Decide if you need to spend money on:

- Domain name (optional, \$10-15/year).
- Cloud hosting credits if free tiers are insufficient.
- Premium API access if free tiers are too restrictive.

My planned budget: _____

Breakdown: _____

If zero budget, state that explicitly and justify free alternatives.

7 Market Data Sources

Instructions: List which free/paid APIs you will use for real-time or delayed stock/crypto data. Consider:

- Yahoo Finance (unofficial APIs, be careful with terms).
- Alpha Vantage (free tier with 5 calls/min).
- Twelve Data, IEX Cloud, or Polygon.io (free tiers available).
- For crypto: CoinGecko or Binance public WebSocket.

Primary API(s): _____

Backup API(s): _____

Rate limits and caching strategy: _____

8 Development Timeline

Instructions: Break down into phases (example below). Adjust weeks based on your schedule.

Phase	Deliverables
Week 1-2	Finalize API choice, setup C++ HTTP server skeleton, basic database schema.
Week 3-4	Implement data fetcher + caching layer. WebSocket broadcast of prices.
Week 5-6	Build React frontend with chart and trade form. Connect to backend.
Week 7-8	Portfolio logic, trade logs, P&L calculations.
Week 9-10	Polish UI, add optional features (alerts, leaderboard).
Week 11-12	Testing, deployment, documentation.

Your actual timeline: _____

(Attach Gantt chart or detailed list if needed)

9 Risks and Mitigation Strategy

Instructions: Identify at least 3 technical or project risks and how you'll address them.

Example:

- **Risk:** C++ web development has a steep learning curve.

Mitigation: Use well-documented frameworks (Drogon or Beast with example projects).

- **Risk:** Free API rate limits cause stale data.

Mitigation: Implement aggressive caching and fallback to delayed data.

Your risks:

1. _____
2. _____
3. _____

10 Success Criteria

Instructions: State what "done" looks like for you. Be specific and measurable.

Examples:

- A user can register, receive virtual cash, buy/sell 5 different stocks, and see updated P&L within 2 seconds of trade.

- The backend handles 100 simulated trades per second without crashing.

- All core requirements from Section 4 are met and demoable.

My success criteria:

1. _____
2. _____
3. _____

Conclusion

QuantSim will serve as a challenging, real-world style portfolio project that demonstrates full-stack proficiency with an emphasis on performance. By using C++ for the backend, I differentiate this project from typical Python-based simulators and prove my ability to build low-latency systems. The modular design allows incremental delivery of core features, with optional AI and automation components as stretch goals.

Prepared by: Aidan Dawson **Date:** 16-04-2026